

Guia Técnico de DevOps: Branching e Correções

Este documento detalha o fluxo completo de criação de branch e correção isolada para cada bug restante listado pelo pytest.

Tarefa 1: Validação do Formulário de Contato

Branch Dedicada: fix/contact-validation

Arquivo a Modificar: app/routers/contact.py

Erro no Pytest: test_empty_contact_returns_400 - assert 201 == 400

Comandos no Terminal para esta tarefa:

```
git checkout main
git checkout -b fix/contact-validation
# [Substitua o código do arquivo contact.py pelo bloco abaixo]
python -m pytest
git add app/routers/contact.py
git commit -m "fix(contact): adiciona validacao de erro 400 para campos vazios"
git push -u origin fix/contact-validation
```

Código Corrigido para app/routers/contact.py:

```
from fastapi import APIRouter, HTTPException, status
from pydantic import BaseModel, Field

router = APIRouter(prefix="/contact", tags=["Contact"])

class ContactSchema(BaseModel):
    name: str = Field(..., min_length=1)
    email: str = Field(..., min_length=1)
    message: str = Field(..., min_length=1)

@router.post("")
def send_contact(contact: ContactSchema):
    if not contact.name.strip() or not contact.email.strip() or not contact.message.strip():
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Campos obrigatorios nao podem estar vazios"
        )
    return {"message": "Contato enviado com sucesso!"}
```

Tarefa 2: Correção dos Status Codes de Cursos

Branch Dedicada: fix/courses-status-codes

Arquivo a Modificar: app/routers/courses.py

Erros no Pytest: Listagem (201 == 200), Busca Inexistente (200 == 404), Exclusão (200 == 204)

Comandos no Terminal para esta tarefa:

```
git checkout main
git checkout -b fix/courses-status-codes
# [Substitua os blocos correspondentes ou ajuste o arquivo inteiro conforme abaixo]
python -m pytest
git add app/routers/courses.py
git commit -m "fix(courses): corrige status codes HTTP de listagem, busca e exclusao"
git push -u origin fix/courses-status-codes
```

Ajustes estruturais sugeridos para app/routers/courses.py:

```
# 1. Decorador da rota de listagem (Garantir status 200)
@router.get("", status_code=status.HTTP_200_OK)
def list_courses(active: bool = True):
    return courses_list

# 2. Rota de Busca por ID (Disparar erro 404 se nao existir)
@router.get("/{course_id}", status_code=status.HTTP_200_OK)
def get_course(course_id: int):
    course = courses_service.find_by_id(course_id)
    if not course:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Curso nao encontrado"
        )
    return course

# 3. Rota de Exclusao (Mudar para status 204 e retornar None)
@router.delete("/{course_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_course(course_id: int):
    # ... logica atual de remocao ...
    return None
```

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.